Embeddings

Lesson 1: Tokenization and Token Embeddings

Agenda

- **Tokenization**
  - **Tokens**: what a model sees?
  - **Tokenization**: From text to tokens
- **Token embeddings**: Providing meaning to words (encoding) - semantics
  - **word2vec**
  - **GloVe**

Learning Objectives

By the end of this lesson, you'll be able to:

- Understand what **tokens** are and how they're generated.
- Understand what a language model actually **sees**.
- **Train** a simple tokenizer.
- Understand how **embeddings** work.

Why this matters

A neural network cannot read words — it reads numbers. Every model in this course, from a tiny skip-gram network to a multi-billion-parameter Transformer, starts by converting your sentence into a sequence of numbers, and *how* it does that shapes everything downstream.

- Cut the text the wrong way, and whole words can silently **disappear** from what the model sees — or a sentence explodes into hundreds of meaningless little pieces.
- Represent words the wrong way, and "similar" words end up looking just as **unrelated** as random ones — everything the model later does with meaning inherits this mistake.
- It's also very practical: the tokenizer decides how many "tokens" your API call is billed for, and why a model fumbles a word it has never seen.

Part I — Tokenization

How can we process text?

A model cannot read words — it reads **numbers**. To pass text into a model, we first cut it into pieces called **tokens**, then turn each piece into a number. Choosing *how* to cut the text — where one token ends and the next begins — is one of the first hard problems in processing text, and it's what we tackle in this part of the lesson.

What exactly is a token?

The definition is deliberately loose: **a token is an arbitrarily defined unit of text**. "Arbitrarily" is the load-bearing word — a token is *not* synonymous with a word, syllable, or characters; it is whatever unit the tokenizer's designer chose.

> The model never sees text at all. It sees a sequence of **integer ids**, one per token — and everything it ever learns about language attaches to those ids.

Picking the unit is therefore the *first* modeling decision of any NLP system — made before any neural network exists.

What is a vocabulary?

Vocabulary V = the fixed set of tokens the model knows — concretely a **bijection** between tokens and ids $\{0, \ldots, |V| - 1\}$, the numbers a tokenizer actually outputs.

Made concrete: one sentence, four granularities

Our running example: `my old paper lantern is glowing`.

Granularity	The same sentence, tokenized	# tokens
Word	`my` `old` `paper` `lantern` `is` `glowing`	6
Subword	`my` `old` `paper` `lant` `##ern` `is` `glow` `##ing`	8
Character	`m` `y` `_` `o` `l` `d` `_` `p` `a` `p` `e` `r` …	31
Byte (UTF-8)	`109` `121` `32` `111` `108` `100` `32` `112` …	31

The `##` prefix marks "continues the previous token". A typo like `lanturn`: word-level → unknown `[UNK]`; subword → decomposes (`lant` `##urn`).

The trade-off

Two quantities — **vocabulary size** and **sequence length** — move in opposite directions as granularity gets finer (byte ← char ← subword ← word):

Granularity	Vocabulary size $\lvert V \rvert$	Sequence length	OOV behavior
Word	very large	shortest	unseen word → `[UNK]` — **severe**
Subword	moderate	short–moderate	decomposes into pieces — **essentially solved**
Character	tiny	long	only unseen *characters* fail
Byte	256	longest	never — all text is bytes

*OOV = **out-of-vocabulary**: what happens when the input contains something the vocabulary lacks — spelled out per granularity next.

Special tokens, and OOV spelled out per granularity

Special tokens live in V alongside content tokens:

- `[PAD]` — padding, filling each sentence up to the batch's maximum length so sequences stack into one tensor;
- `[UNK]` — *unknown*: stands in for any piece of input not in the vocabulary.

Out-of-vocabulary (OOV) behavior — what happens when the input contains something the vocabulary lacks:

- *word-level*: the **whole word** collapses to `[UNK]` — meaning destroyed silently on any text with new words, names, or typos.
- *subword*: the word **decomposes** into known pieces, at worst individual characters — graceful degradation;
- *character | byte-level*: essentially never fails.

Why subwords win

✅ Leverage root meanings

`run`, `runner`, `running` share the piece `run` — the model reuses what it learned about one for the others.

✅ Reduce OOV occurrences

`lanterns` → `lantern` + `s`, instead of a brand-new unknown word.

⚠️ Misconception: "Subword tokenizers understand grammar." They don't — pieces are chosen by *statistical frequency*; `token` + `ization` splitting cleanly is a happy accident, not a linguistics engine.

⚠️ Misconception: "Token counts are universal." Every model family has its own tokenizer — the same sentence yields different token counts under GPT, BERT, or LLaMA.

Tokenizer

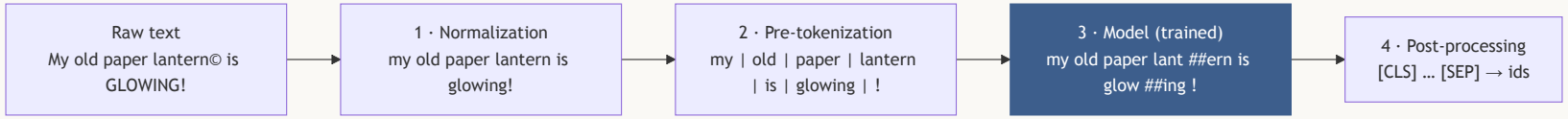
A tokenizer is a translator with a phrasebook

A **tokenizer** T converts input text into tokens so they can later be represented as vectors. It runs in two directions:

- **Encode**: text → tokens → ids — what the model consumes.
- **Decode**: ids → tokens → text — needed whenever the model *produces* tokens (generation) and a human must read the result.

Production tokenizers do this as an **assembly line with four stations**: clean the text, chop it into word-like chunks, split the chunks into vocabulary pieces, then package the result with special tokens. Only the third station is *learned*; the rest are fixed rules.

The pipeline, stage by stage



1. **Normalization** — standardize the raw string (casing, accents, Unicode cleanup) so the vocabulary isn't wasted on cosmetic variants.
2. **Pre-tokenization** — split on whitespace/punctuation into word-like chunks; an **upper bound** on the final tokens (the learned stage only ever splits *within* them).
3. **Model** — the **trained** subword segmenter (**BPE** = Byte-Pair Encoding, **WordPiece**, **Unigram**) — the only stage that requires training.
4. **Post-processing** — add the model-specific special-token template and emit the final id sequence.

Encode ↔ decode, and when the round trip is lossy

Decoding inverts the pipeline: ids → tokens (vocabulary lookup), strip special tokens, undo continuation markers (`glow` + `##ing` → `glowing`), re-insert spaces.

> ⚠️ But $\mathrm{decode}(\mathrm{encode}(x))$ returns the **normalized** text, not x — our example decodes to `my old paper lantern is glowing!`, with the capitals and `©` gone forever — and anything that hit `[UNK]` is irrecoverable.

SentencePiece (coming up) is engineered to make this round trip **lossless**: spaces become the visible symbol `▁` and are kept inside the tokens themselves.

Rule-based vs. learned tokenizers

Rule-based

- word- and character-level splitting
- need **no training**
- don't exploit patterns in the data
- typically less optimal results

Learned

- BPE, WordPiece, Unigram
- **trained** on a corpus $\mathcal{D}$, reflecting patterns actually present
- performs better

Two clocks. Tokenizer *training* happens once, before the model exists — pure counting / likelihood fitting, no gradients. Afterwards the tokenizer is **frozen**: at model-training time and at inference it only *applies* its stored rules.

Three algorithms, one goal

BPE and WordPiece **build up**: start from characters and greedily glue the most useful adjacent pair. Unigram **tears down**: start from a huge candidate set and prune what the corpus can most afford to lose. All three reach the same destination — a fixed-size vocabulary that covers the corpus with few, reusable pieces.

Algorithm	Direction	Training criterion	Encoding rule	Powers
BPE	build up	merge the **most frequent** adjacent pair	replay merges in learned order	GPT, LLaMA
WordPiece	build up	merge the pair with the highest **likelihood gain**	greedy longest-match	BERT
Unigram	tear down	prune pieces that least hurt corpus likelihood	most-probable segmentation	T5, ALBERT (via SentencePiece)

A brief history

- **1994** Byte-Pair Encoding — a data-*compression* trick (Gage [3])
- **2012** WordPiece — Google voice search (Schuster & Nakajima [6])
- **2016** BPE adapted for neural machine translation (Sennrich et al. [4])
- **2018** Unigram LM + SentencePiece (Kudo [9]; Kudo & Richardson [10])
- **2019** Byte-level BPE — GPT-2 closes the last OOV hole (Radford et al. [5])

BPE: merge what is frequent

BPE (*Byte-Pair Encoding*) constructs the vocabulary by learning from the most common pairs of entities in the training corpus. It began life as a 1994 data-compression trick (Gage [3]) and was imported into NLP for machine translation by Sennrich et al. [4].

Step 1 — initialize. Split every word into characters. Training corpus: `a paper lantern is glowing over the other lanterns`. The initial vocabulary is the unique characters `{a,e,g,h,i,l,n,o,p,r,s,t,v,w}`, size $v_i = 14$.

Pair counting happens *within* pre-tokenized words (pipeline stage 2) — never across spaces.

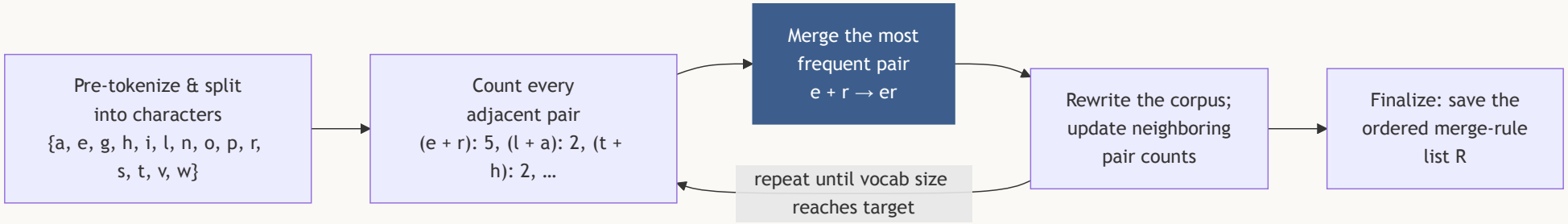
BPE: count every adjacent pair

Step 2 — count. First iteration:

pair	frequency	where
`e·r`	5	paper, lantern, over, other, lanterns
`l·a`, `a·n`, `n·t`, `t·e`, `r·n`	2	lantern, lanterns
`t·h`, `h·e`	2	the / other, the / other
every other pair	1	—

`e·r` wins outright with count 5.

BPE: merge and iterate



Step 3 — merge the winner: add `er` to V, rewrite every `e r` → `er`. Only counts of pairs *touching* a merged occurrence change (e.g. `h·e` drops from 2 to 1; new pairs `t·er`, `er·n`, `p·er` appear) — the bookkeeping trick that keeps real trainers fast.

Step 4 — iterate: the next round is a **6-way tie at count 2**. Real implementations break ties with a fixed, deterministic rule. Following the `lantern` chain, successive merges `la → lan → lant → lanter → lantern` collapse the word into a **single token** — after which `lanterns` costs only one extra symbol, `lantern` + `s`.

BPE: finalize and encode

Step 5 — finalize when $|V|$ reaches the target v_f. The trainer saves the **ordered merge-rule list** $R = \langle (\texttt{e}, \texttt{r}), (\texttt{l}, \texttt{a}), (\texttt{la}, \texttt{n}), \ldots \rangle$ — the merge list *is* the tokenizer.

Encoding new text — split into characters and **replay the merge rules of R in the order they were learned**, applying each everywhere it fits, until none apply. No counting happens at encoding time; the corpus statistics were frozen into R at training time.

On `the old paper lanterns`: `e·r` → `er` first, then the `lantern` chain, then `t·h` → `th`, … → final tokens:

`th · e · o · l · d · p · a · p · er · lantern · s`

BPE: OOV handling and byte-level BPE

Every word decomposes at worst into characters, so `[UNK]` survives only for *characters* absent from training (a never-seen script or symbol).

> **Byte-level BPE** (GPT-2 [5] and most LLMs since) closes even that hole: take the 256 possible bytes as the base alphabet and merge upward from there. Every string is a byte sequence, so *nothing* is ever OOV — by construction.

Practical footnote: real BPE distinguishes word-boundary from word-internal pieces (Sennrich's `</w>` suffix, GPT-2's `Ġ` space-prefix, SentencePiece's `_`), so `er` ending a word and `er` mid-word can be different tokens. Our toy example drops this marker for clarity.

BPE: strengths & limitations

- ✅ Every word decomposes at worst into characters — `[UNK]` survives only for unseen characters.
- ✅ Byte-level BPE closes even that hole — nothing is ever OOV.
- ⚠️ Merges are chosen by **frequency alone**: `token` + `ization` is a statistical accident, and "junk" merges like `t·h` happen just as readily — the flaw WordPiece's score targets next.
- ⚠️ Vocabulary size is a hyperparameter: too small → long sequences; too large → rare pieces whose embeddings barely train (more on this later this lesson). Typical $|V|$ is 30k–100k.

WordPiece: merge what sticks together

WordPiece runs BPE's exact build-up loop with one line changed — the *score*. BPE asks "which pair occurs most often?"; WordPiece asks "which pair occurs **more often than chance**?" — the difference between a *popular* pair and an *inseparable* pair.

`t·h` is frequent largely because `t` and `h` are individually everywhere; `q·u` is glued because `q` essentially never appears without `u`. WordPiece is a **collocation detector**.

WordPiece: the score, derived

At each iteration WordPiece merges the pair maximizing:

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a)\,\text{freq}(b)} \tag{1}$$

Model the segmented corpus with a unigram language model: each piece u has probability $P(u) = \text{freq}(u)/N$. Merging $a, b \to ab$ changes the corpus log-likelihood by, per occurrence:

$$\log \frac{P(ab)}{P(a)\,P(b)} = \log \frac{\text{freq}(ab) \cdot N}{\text{freq}(a)\,\text{freq}(b)} \tag{2}$$

Since N (unigrams) is the same for every candidate pair, ranking by (2) is ranking by (1) — this quantity is exactly the **pointwise mutual information (PMI)** of a and b.

WordPiece: same corpus, different winner

Character frequencies in `a paper lantern is glowing over the other lanterns`: $e{=}6, r, n{=}5, a, t{=}4, l, o{=}3, p, h, i, s, g{=}2, w, v{=}1$.

pair	$\text{freq}(ab)$	$\text{freq}(a)$	$\text{freq}(b)$	score (1)
`e·r` — *BPE's winner*	5	6	5	0.17
`l·a`	2	3	4	0.17
`t·h`	2	4	2	0.25
`o·v`	1	3	1	0.33
`w·i`	1	1	2	**0.50**

WordPiece's first merge is `w·i`: `w` occurs once and is *always* followed by `i` — perfect association. And `t·h` (0.25) beats BPE's `e·r` (0.17), because `e` and `r` are so individually common that much of their pairing is coincidence.

WordPiece: continuation marks and encoding

The `##` continuation convention. WordPiece vocabularies distinguish *word-initial* pieces from *continuation* pieces: `lant` vs `##ern`. The `##` prefix (BERT's choice [7]) means "glue me to the previous piece"; it's stripped at decode time.

Encoding is greedy longest-match, *not* merge replay:

1. Find the **longest prefix** of the word that is in V; emit it.
2. Prepend `##` to the remainder and repeat on it.
3. If at any step **no prefix matches**, emit `[UNK]` for the **entire word** and stop.

Worked: `lanterns` with $V \supseteq \{\texttt{la}, \texttt{lant}, \texttt{lantern}, \texttt{\#\#ern}, \texttt{\#\#s}\}$ → longest match `lantern`, remainder `s` → `##s` $\in V$ → `lantern · ##s`.

WordPiece: strengths & limitations

⚠️ **Worked failure.** `lanternz` — `lantern` matches, but if `##z` $\notin V$, the **whole word** becomes `[UNK]`, even though seven of its eight characters were representable. Contrast BPE, which falls back to characters and loses only the `z`. This whole-word `[UNK]` is real, observable BERT-tokenizer behavior.

- ✅ Ranks by statistical association, not raw count — filters out "popular but coincidental" pairs.
- ⚠️ Its whole-word `[UNK]` rule is locally *worse* than BPE's character fallback.
- ⚠️ Discards the merge history entirely — a genuinely different encoder, not just a different score.

Unigram: keep what the corpus needs

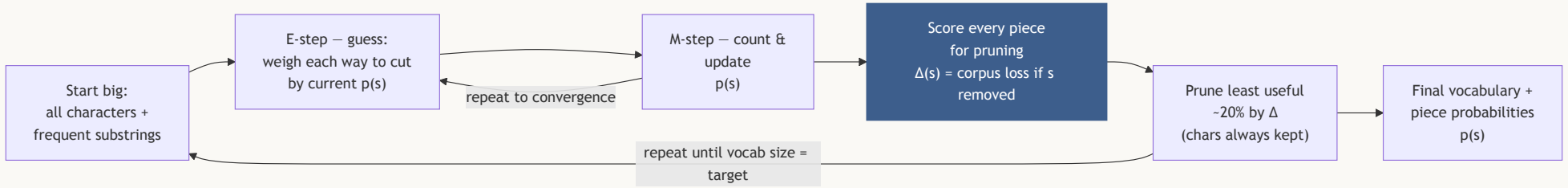
BPE and WordPiece are *bricklayers* — they build the vocabulary up, piece by piece. **Unigram** is a *sculptor* — it starts from a block far too big and carves away what the corpus doesn't need.

The intuition:

1. Start with a huge candidate vocabulary — every character, plus lots of common substrings.
2. Give every piece a score for how useful it is to the corpus $p(s)$.
3. Repeatedly ask: **"if I removed this piece, how much would the corpus miss it?"** $\Delta(s)$ — Barely missed → cut it. Really needed → keep it.
4. Keep shrinking the vocabulary this way until it hits the target size.

> Where do the scores come from? That's the training loop, next.

Unigram: the training loop



1. **Start big:** seed with every character plus lots of frequent substrings — deliberately over-large.
2. **Estimate each piece's probability:** for every word, look at all the ways it could be cut, weigh each way by how likely is, and count how often each piece gets used. Update every piece's probability to match those counts. Repeat this cycle until the numbers settle. *(This loop has a name — Expectation–Maximization* $\Delta(s) = \mathcal{L}(V \setminus \{s\}) - \mathcal{L}(V)$*.)*
3. **Score every piece:** how much would the corpus miss it if it were gone, $\Delta(s)$?
4. **Prune the least useful ~20%** by Δ, **never removing single characters** — every word stays tokenizable.

Unigram: pruning in numbers

A large Δ means "the corpus needs this piece." On our running corpus, removing the workhorse `er` hurts far more than removing `la`:

piece s removed	$\Delta(s) = \mathcal{L}(V \setminus \{s\}) - \mathcal{L}(V)$
`er`	**4.10**
`la`	1.85

(illustrative numbers)

> Pieces with **near-zero** Δ are the first to be pruned — the corpus barely notices they're gone.

Unigram: Encoding (1/2)

Once each piece has a probability, encoding is simple: **try every way to cut the word, and pick the most probable one.** Assume a segmentation's pieces are chosen independently, so its probability is just the product of its pieces' probabilities.

$$P(\mathbf{s}) = \prod_{k=1}^{K} p(s_k), \qquad \sum_{s \in V} p(s) = 1 \quad (3)$$

Take `paper` with this (illustrative, EM-fitted) piece table:

piece s	`p`	`a`	`e`	`r`	`er`	`pa`
$p(s)$	0.10	0.12	0.10	0.09	0.05	0.04

Unigram: Encoding (2/2)

All four segmentations, scored with (3):

segmentation	probability
`p·a·p·e·r`	1.08×10^{-5}
`pa·p·e·r`	3.60×10^{-5}
`p·a·p·er`	6.00×10^{-5}
`pa·p·er`	$\mathbf{2.00 \times 10^{-4}}$ ✓ winner

Two lessons hide in the arithmetic: (i) **fewer pieces usually win** — every extra factor is < 1; (ii) but only if those pieces are *probable* — length and frequency trade off automatically.

Choosing among the three

All three solve OOV and converge to broadly similar vocabularies at scale; the choice in practice follows ecosystem lineage — GPT-line → byte-level BPE, BERT-line → WordPiece, T5/multilingual → Unigram.

Algorithm	Criterion, in one phrase
BPE	raw frequency
WordPiece	association strength (PMI)
Unigram	corpus likelihood under a generative model

Three different answers to the same question: **what makes a piece worth a vocabulary slot?**

→ To the lab: subword tokenizers

In the in-class lab's first mini-lab, you'll **train a BPE tokenizer and a Unigram tokenizer on the same corpus**, watch a made-up word survive without becoming `[UNK]`, and read a Unigram tokenizer's learned piece log-probabilities directly out of its saved model — the same $p(s)$ from equation (3), on real data.

Part II — Token Embeddings

A vocabulary isn't a language

Tokenization (Part I) solved one problem: it gave the model a fixed **vocabulary** — a closed set of symbols, each with its own id. But that's just a list. Nothing about it says which words are related, similar, or even about the same topic — `paper`, `lantern`, and `airplane` are just three different integers so far.

Two things are still missing:

- **Meaning.** A language isn't a list of words — it's the *relationships* between them. "Similar" words should end up close together, unrelated words far apart.
- **A number a model can compute with.** To do math on text, every token needs a numeric structure that actually *represents* something — not just an arbitrary index.

> **Embeddings** are how we get both at once: turn each token id into a vector a model can compute with, whose geometry itself encodes meaning. That's the rest of this lesson.

One-hot encoding (1/2)

The simplest way to turn a token id into a vector is a **one-hot** vector: a vector as long as the vocabulary, with a single 1 marking that token's position and 0s everywhere else. It correctly identifies *which* word it is, but it carries no information about *meaning* — every pair of distinct words ends up exactly the same distance apart, no matter how related they actually are.

Concrete example: with $V = \{\texttt{a}, \texttt{cute}, \texttt{paper lantern}, \texttt{airplane}, \ldots\}$, `cute` $= (0, 1, 0, 0, \ldots)$ and `paper lantern` $= (0, 0, 1, 0, \ldots)$ — the two vectors share no nonzero position, so they have **nothing in common**. The same is true for `airplane` and `paper lantern`.

> One-hot says all three words are **equally unrelated** — `cute` is just as "different" from `paper lantern` as `airplane` is. Meaning needs *geometry*, and one-hot has none.

One-hot encoding (2/2)

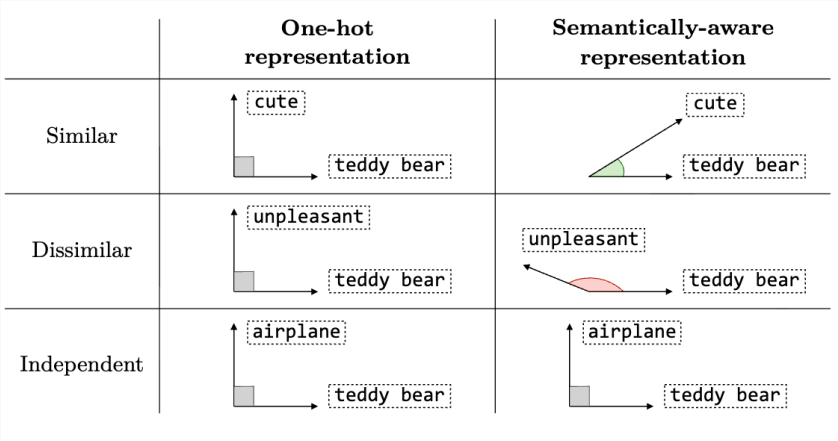
In symbols: token w_i becomes the vector e_i — all zeros, except a single 1 at position i.

$$x_{\mathrm{OHE}}(w_i) = e_i \in \{0,1\}^{|V|}, \qquad (e_i)_k = \mathbf{1}[k = i]$$

Two fatal limitations, in plain terms first:

1. **No similarity structure.** Every pair of distinct words sits at *exactly* the same distance from each other — synonyms and total opposites alike. *(In symbols: their dot product $\langle e_i, e_j \rangle = 0$ and their distance $\|e_i - e_j\| = \sqrt{2}$, for any two distinct tokens, always.)*
2. **It's huge.** The vector's length equals the size of the vocabulary — tens of thousands of numbers, almost all zero, just to name *one* word.

One-hot vs. a meaning-aware representation



- **One-hot (left):** always 90° — every pair looks equally unrelated.
- **Meaning-aware (right):** the angle *reflects* the relationship — small when similar, large when dissimilar, ~90° when unrelated.

> This is the geometry embeddings are built to produce — the next slides show how.

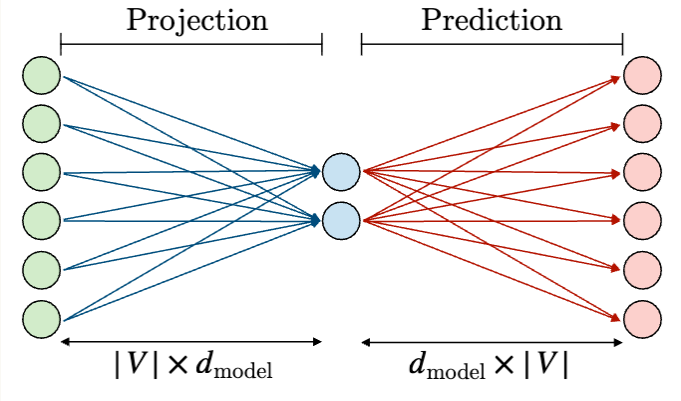
The fix: a dense, learned vector (1/2)

Replace the one-hot with a short **dense** vector $v_w \in \mathbb{R}^d, d \sim 10^2$–$10^3$, whose coordinates are *learned* so that geometric closeness means semantic closeness.

Picking row i of an embedding matrix $E \in \mathbb{R}^{|V| \times d}$ is exactly a matrix–vector product:

$$E^\top x_{\text{OHE}} = v_{w_i}$$

The fix: a dense, learned vector (2/2)



Only the **left half** ("Projection") matters for now: green nodes = the one-hot's slots, the blue fan = E, the 2 blue nodes = v_{w_i}. *(The right half is word2vec — later.)*

An embedding **lookup is one-hot × matrix** — which is why E is also called the *projection* matrix.

From one token to a whole sentence

A sentence is a **sequence** of ids $(i_1, \ldots, i_T)$ — look up each row in E and **stack** them, in order.

Concrete example: `my old paper lantern is glowing` — $T = 6$ tokens, stacked top to bottom:

$$\begin{pmatrix} v_{\text{my}} \\ v_{\text{old}} \\ v_{\text{paper}} \\ v_{\text{lantern}} \\ v_{\text{is}} \\ v_{\text{glowing}} \end{pmatrix} \in \mathbb{R}^{6 \times d}$$

Six rows in, one $T \times d$ matrix out. **This matrix — not the text, not the one-hots — is what every model in this course consumes from here on.**

The gradient: only used rows learn

A row of E only gets adjusted if that word actually showed up in the training example just processed — every other row stays **completely untouched** for that step.

- **Common words** appear in thousands of sentences → thousands of tiny nudges over training → end up well-trained.
- **Rare words** appear only a handful of times → a handful of nudges → barely move from where they started (randomly) → end up poorly trained.

> ⚠️ This is the reason an oversized vocabulary hurts: it's full of rows that get touched so rarely they never really learn anything.

Where the numbers come from: two routes

Pretext-task pretraining

- word2vec / GloVe (coming up) train E on raw text and throw everything else away

End-to-end

- initialize E randomly
- shaped by the *downstream task's* backprop, jointly with the rest of the network
- how a Transformer's embedding table is trained (a later module)

Either way, E is ordinary trainable parameters — there is no hand-designed feature anywhere in it.

What the vectors capture — and what they don't

Every training signal above sees only *which tokens occur in which contexts* — so the geometry of E can encode exactly that, and nothing more. This is the **distributional hypothesis** (Harris [11]; Firth [12]).

> **Meaning lives in relative geometry, not in coordinates.** No single axis of E has a fixed meaning on its own — training only ever compares vectors to each other, never to a fixed axis. So "dimension 7" is not "the gender dimension"; only **relative positions and directions** (like $v_{\text{queen}} = v_{\text{king}} - v_{\text{man}} + v_{\text{woman}}$) carry meaning.

This also bounds what embeddings *cannot* know: anything the co-occurrence statistics don't express (perception, ground truth, events after the training corpus).

Nearest neighbors, made concrete

Example:

Query	Neighbor	Cosine similarity
`king`	`prince`	0.824
`king`	`queen`	0.784
`king`	`emperor`	0.774
`paper lantern`	`lamp`	0.663

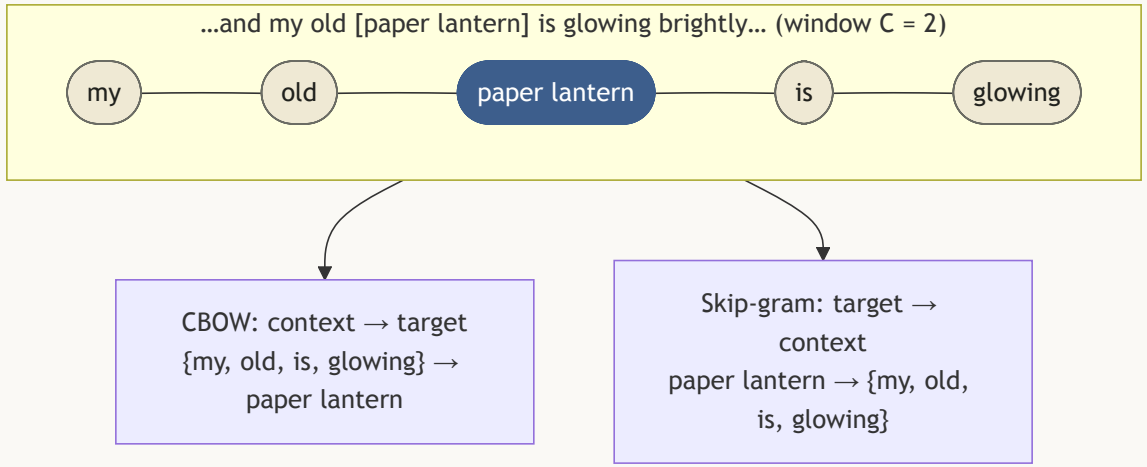
(cosine similarity itself — "how aligned are two vectors" — gets a full treatment in a later lesson; for now, read it as "higher = more related.")

Embedding matrix: strengths & limitations

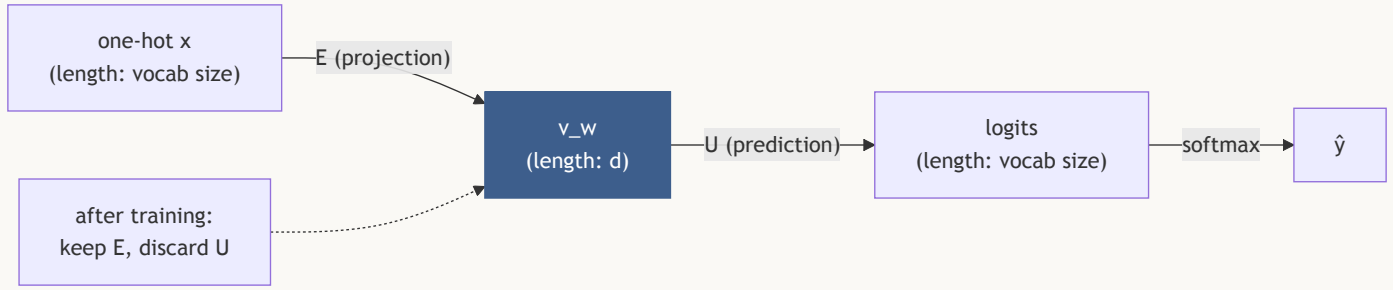
- ✅ Ordinary trainable parameters — no hand-designed features anywhere.
- ✅ The same table underlies everything downstream: E is literally **layer 0 of every LLM**.
- ⚠️ **Static, not contextual**: one row per token means `bank` (river) and `bank` (money) collapse to a single point — fixed later by *contextual* embeddings (a forward pointer to a later lesson).
- ⚠️ **One vector per token**, not per word — subword pieces get vectors too; a rare word's representation is *assembled* from its pieces.

word2vec: predict the company a word keeps

word2vec [13] turns this into a training game: slide a window along raw text and repeatedly ask a tiny network to **predict a word from its neighbors** (CBOW, short for *continuous bag-of-words*) or **its neighbors from a word** (skip-gram). To win, the network is *forced* to place words that appear in similar contexts near each other. The prediction task is a pretext — the **weights are the prize**.



Architecture: a shallow network, two matrices



A shallow network, one hidden layer, **no nonlinearity** in it [1]:

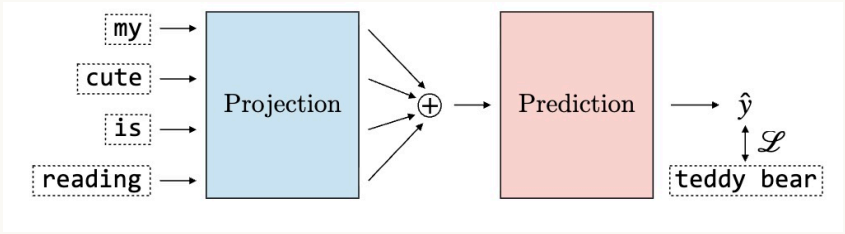
$$x_{\text{OHE}} \xrightarrow{E\ (\text{"projection"})} v_w \xrightarrow{U\ (\text{"prediction"})} \text{logits} \xrightarrow{\text{softmax}} \hat{y}$$

After training, **throw away U and keep E** — its rows are the token embeddings.

CBOW vs. skip-gram

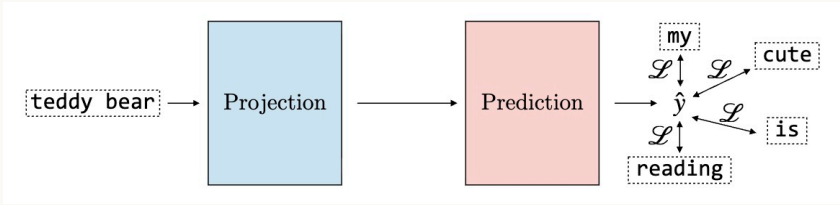
Same shallow network, same **Projection**/**Prediction** shape — only which side is "one word" and which side is "many words" flips.

CBOW: context → target



Several context words go in, get summed; the network predicts the **one** word in the middle.

Skip-gram: target → context



One center word goes in; the network predicts each context word separately.

The skip-gram objective (and its bottleneck)

For a center token w (input embedding v_w, a row of E) and a context token o (output embedding u_o, a row of U), skip-gram turns their similarity into a probability with a softmax over the **whole vocabulary**:

$$P(o \mid w) = \frac{\exp\left(u_o^\top v_w\right)}{\sum_{w' \in V} \exp\left(u_{w'}^\top v_w\right)} \tag{4}$$

Training minimizes the negative log-likelihood of this probability, summed over every (center, context) pair in the corpus.

⚠ **The bottleneck**: that denominator sums over **all** $|V|$ tokens for *every* training pair. With $|V| = 50{,}000$, that's 50,000 dot products per step — too slow to train at scale. Negative sampling, next, fixes exactly this.

A cheaper question: real or fake?

Reframe the expensive $|V|$-way softmax as a cheap **binary** question: "is this (center, context) pair *real* or *fake*?"

- **Positive example:** the true pair (w, o) — label 1.
- **Negative examples:** draw $k \approx 5$–20 "noise" tokens $w_n \in \mathcal{N}$ that are *not* the context — label 0.

Negative sampling, the objective

$$\mathcal{L}_{\text{NS}} = -\log \sigma\big(u_o^\top v_w\big) \; - \sum_{w_n \in \mathcal{N}} \log \sigma\big(-\, u_{w_n}^\top v_w\big) \tag{5}$$

1. $\sigma(u_o^\top v_w)$ = "probability this pair is real" (want $\to 1$).
2. $\sigma(-u_{w_n}^\top v_w) = 1 - \sigma(u_{w_n}^\top v_w)$ = "probability this fake pair is fake" (want $\to 1$).
3. Take $-\log$ of the product of these Bernoulli likelihoods → (5). This is exactly the source's boxed loss.
4. **Cost:** $k+1$ terms instead of $|V| - 50{,}000 \to \sim 6$: orders of magnitude cheaper, for a "good enough" approximation.

Noise distribution, and CBOW's mirror

Noise distribution. Negatives are sampled not uniformly, but from the **unigram frequency raised to the $3/4$ power**, $P_n(w) \propto U(w)^{3/4}$ — up-samples rare words and empirically gives the best embeddings (Mikolov et al. [14]).

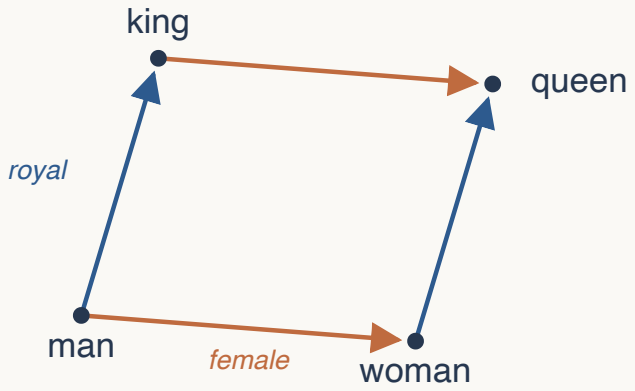
> **CBOW** mirrors this with the roles swapped: average the $2C$ context vectors, $\bar{v} = \frac{1}{2C}\sum_c v_c$, then predict the center token via the same softmax/negative-sampling head. One prediction per window → faster training than skip-gram; skip-gram usually wins on rare words and small data.

The payoff: geometry and analogies

Because contexts shape the vectors, semantic relationships become **directions** in the space:

$$v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$$
$$v_{\text{lamp}} - v_{\text{glass}} + v_{\text{paper}} \approx v_{\text{paper lantern}} \quad (6)$$

Read the first: "king is to man as ? is to woman" is answered by *vector arithmetic* — the `royal` direction plus the `female` direction.



word2vec: strengths & limitations

- ✅ Learns purely from **local context** — the words inside a sliding window — turning raw, unlabeled text into a self-supervised training signal; no human labels needed.
- ✅ Analogies emerge as **linear structure**, a real and provable consequence of the objective, not a coincidence.
- ⚠️ Keeps the **input/projection** matrix E; the output head U is scaffolding, thrown away.
- ⚠️ Only ever sees a **local** window (size C) per update — never the corpus's global co-occurrence statistics. GloVe, next, fixes exactly this.

GloVe: a global counterpoint

word2vec learns from **local** windows, one at a time. **GloVe** [16] ("Global Vectors") instead reads the **whole corpus once** into a co-occurrence table — how often does word i appear near word j? — and then fits vectors so their dot product reproduces the *log* of those counts.

> Same destination — a geometric space of word meaning — different route: global statistics instead of local prediction.

GloVe, the math (1/2)

1. **Count once, over the whole corpus.** Build a symmetric **co-occurrence matrix** X: X_{ij} = how often context word j occurs within target word i's window — tallied across every sentence.
2. **Fit vectors to match those counts.** Give every word two roles — a **target** vector w_i and a **context** vector $\tilde{w}_j$ — plus one scalar bias each, b_i and $\tilde{b}_j$. Choose them so their dot product (plus the biases) approximates the *log* of the count you just measured:

$$\log(X_{ij}) \approx w_i^\top \tilde{w}_j + b_i + \tilde{b}_j \tag{7}$$

The bigger the true co-occurrence count, the bigger this dot product has to be — so words that co-occur a lot end up with aligned vectors.

GloVe, the math (2/2)

3. **Fit by weighted least-squares.** For every pair, compare the model's prediction to the true log-count and square the error — standard regression. But weight each pair's error by $f(X_{ij})$, so extremely frequent pairs (`the`, `a`, …) don't dominate training, and extremely rare (noisy) counts barely count either:

$$\mathcal{L} = \sum_{i,j} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

$$f(x) = \min\left(1,\ (x/X_{\max})^\alpha\right) \tag{8}$$

4. **One vector per word — done $|V|$ times.** Every word ends training with **two vectors of its own**. For each of the $|V|$ words, average *its* pair, $w = (w_i + \tilde{w}_i)/2$ → one final d-dim vector per word ($|V|$ pair $(w_i, \tilde{w}_i)$) of d-dim vectors are trained). The two roles are symmetric, so neither is more "correct."

GloVe, zoomed out: it's matrix factorization

Stack every target vector as a row of a table $W \in \mathbb{R}^{|V| \times d}$, every context vector as a row of $\tilde{W} \in \mathbb{R}^{|V| \times d}$ — **one row per word**, each row a d-dim vector. *(The index i is a **vocabulary id** — it picks which word / which row, never a coordinate inside the vector.)* Then equation (7), for **all pairs at once**, is a single matrix product:

$$\log X \;\approx\; W\,\tilde{W}^{\top}$$

> A $|V| \times d$ times a $d \times |V|$ can only be a **rank-d** matrix — but real $\log X$ has far higher rank, so the fit *can't* be exact: training settles for the best rank-d **approximation**. With only d numbers to explain its whole co-occurrence row, each word is **forced** to share structure with words that behave like it — that squeeze is what creates meaning. *(Same low-rank idea as PCA/SVD.)*

In numbers: $|V|{=}50{,}000$, $d{=}100$ → $\log X$ has ~ 2.5 billion cells; W and $\tilde{W}$ together only ~ 10 million numbers — **~250× smaller**.

→ To the lab: train your own word2vec

In the lab's second mini-lab you'll **build skip-gram (center, context) pairs, implement the negative-sampling loss by hand — equation (5), verbatim — and train a tiny word2vec model**, watching the loss fall from its theoretical starting point (≈ 4.16) toward ≈ 2. You'll then extract the embedding matrix E.

Recap

- **Text → a fixed subword vocabulary.** One goal (no `[UNK]`, no vocab explosion), three criteria: **BPE** = most-frequent pair · **WordPiece** = strongest-association pair · **Unigram** = prune to what a probabilistic model keeps.
- **Each token id → a row of a learned matrix E** — the lookup everything downstream starts from; d is fixed, independent of $|V|$.
- **Two ways to shape those rows: word2vec** (cheap *local* prediction — skip-gram + negative sampling) vs. **GloVe** (fit *global* co-occurrence counts — matrix factorization). Same destination.
- **Meaning lives in relative geometry** (directions/distances, never a single coordinate) — and it's all **static**, one vector per token: ELMo (contextual vectors from bidirectional LSTMs, next lesson) and then attention (Module 2) fix that.

Concept check: two tokenizers (BPE, WordPiece) on the same corpus both avoid `[UNK]` on a rare word — yet split it *differently*. Why? And: why does a token that never appeared in training fail to learn

What's next

Next lesson picks up where the embedding matrix leaves off: how do we turn a whole *sequence* of token embeddings into a single **document embedding**? We'll start with cheap heuristics (bag-of-words, TF-IDF, mean-pooling), then build up through **recurrent encoders** — RNNs (Recurrent Neural Networks) — through the vanishing-gradient problem to **LSTM** (Long Short-Term Memory), to a first look at **attention as weighted pooling** — the seed of everything the rest of this course builds on.

Curious now? Jay Alammar's *Illustrated Word2Vec* is a great visual companion to today's derivations, and Sennrich et al. [4] is worth reading in full for how BPE was adapted from compression to translation.

References & further reading (1/2)

- [1] A. Amidi and S. Amidi, *Super Study Guide: Transformers and Large Language Models*, Part 2 "Embeddings," 2024 — primary source for this lesson.
- [2] Hugging Face, "Tokenizers documentation — The tokenization pipeline," huggingface.co/docs/tokenizers/pipeline.
- [3] P. Gage, "A New Algorithm for Data Compression," *C Users Journal*, 1994.
- [4] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proc. ACL*, 2016 (arXiv:1508.07909).
- [5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners" (GPT-2 technical report), OpenAI, 2019.
- [6] M. Schuster and K. Nakajima, "Japanese and Korean Voice Search," in *Proc. ICASSP*, 2012, pp. 5149–5152.
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional

References & further reading (2/2)

- [9] T. Kudo, "Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates," in *Proc. ACL*, 2018 (arXiv:1804.10959).
- [10] T. Kudo and J. Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing," in *Proc. EMNLP (System Demonstrations)*, 2018 (arXiv:1808.06226).
- [11] Z. S. Harris, "Distributional Structure," *Word*, vol. 10, no. 2–3, pp. 146–162, 1954.
- [12] J. R. Firth, "A Synopsis of Linguistic Theory 1930–1955," in *Studies in Linguistic Analysis*, Blackwell, 1957.
- [13] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," in *Proc. ICLR Workshop*, 2013 (arXiv:1301.3781).
- [14] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and Their Compositionality," in *Proc. NeurIPS*, 2013 (arXiv:1310.4546).